Porto. TechHub

P.PORTO

AI-POWERED SOFTWARE ENGINEERING

| INTRODUÇÃO QUALITY ASSURANCE |

VITOR MANUEL SANTOS

ÍNDICE

- Enquadramento da qualidade em Engenharia de Software
- TDD e BDD
- Software Quality Assurance
  - Categorias de Trabalho SQA
  - Quality Control
  - Gestão da Configuração
  - Baseline
- Testes de Software
  - Diferentes tipos de testes
  - Plano e estratégias de testes
  - Testes unitários

ENQUADRAMENTO DA QUALIDADE EM ENGENHARIA DE SOFTWARE

VITOR MANUEL SANTOS

QUALIDADE DE SOFTWARE

Não se trata apenas de programar. Trata-se de saber:

- o que programar
- como programar
- para quem e com quem
- quando disponibilizar e com que frequência
- como gerir, medir e manter produtos de software

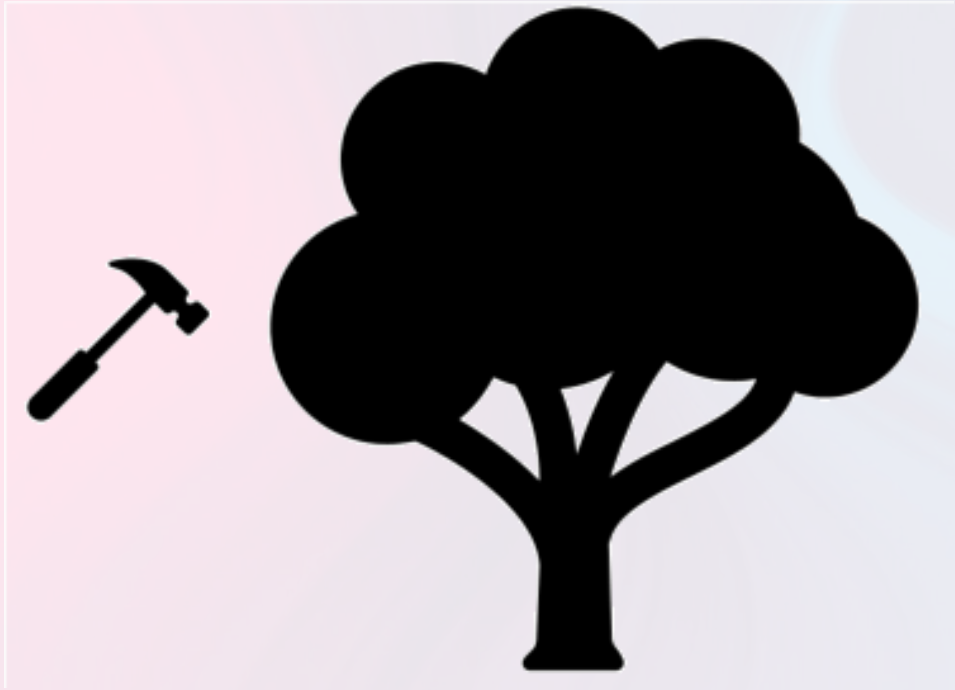
- Trata-se de produzir produtos de software de elevada qualidade.
- Trata-se de aplicar engenharia ao desenvolvimento de software.

QUALIDADE DE SOFTWARE

Imaginemos que há uma árvore que precisamos de cortar.

Podemos cortá-la com um martelo?

Sim, mas…



QUALIDADE DE SOFTWARE



QUALIDADE DE SOFTWARE

- Não seria melhor com um machado?





QUALIDADE DE SOFTWARE

Na prática, para cortar uma árvore:

- usar o equipamento de segurança adequado
- adquirir cunhas de abate
- estimar a zona de queda
- limpar a zona de corte
- avaliar a árvore
- conhecer a anatomia de um entalhe correto
- planear o entalhe
- executar o entalhe
- por exemplo, um corte em V

QUALIDADE DE SOFTWARE



QUALIDADE DE SOFTWARE

Ou seja, para realizar corretamente uma tarefa é necessário:

- perceber o que é necessário para a executar
- planear de acordo com o contexto
- selecionar as ferramentas certas
- possuir as competências adequadas
- combinar estes elementos de forma eficiente e avaliar periodicamente o progresso das ações

QUALIDADE DE SOFTWARE

Em resumo, precisamos de:

- um processo — o que fazer
- um modelo e um método — como fazer
- uma ferramenta — o suporte adequado para fazer com eficiência

PROCESSO

Conjunto de atividades, normalmente agrupadas em fases, com uma determinada sequência lógica, executadas de forma sistemática e uniforme, por pessoas com papéis bem definidos, de modo que, a partir de um conjunto de entradas, se obtenha um conjunto de saídas.



PROCESSO

O processo:

- define o que é necessário fazer (WHAT / O quê), sem especificar como (HOW / Como)
- apresenta uma estrutura com diferentes níveis de agregação
- permite análise e definição em diferentes níveis de detalhe
- apoia diferentes necessidades de decisão



MÉTODO

- Conjunto de técnicas para executar uma determinada tarefa
- Especifica como uma tarefa é executada
- Incorpora em si próprio um processo.

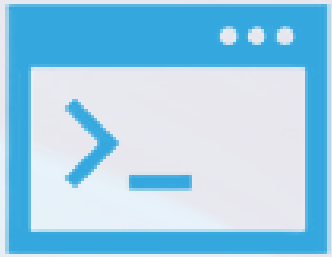


FERRAMENTA

- Instrumento que, quando aplicado a um método específico, aumenta a eficiência de uma tarefa
- Facilita a execução das tarefas, isto é, o como/how.

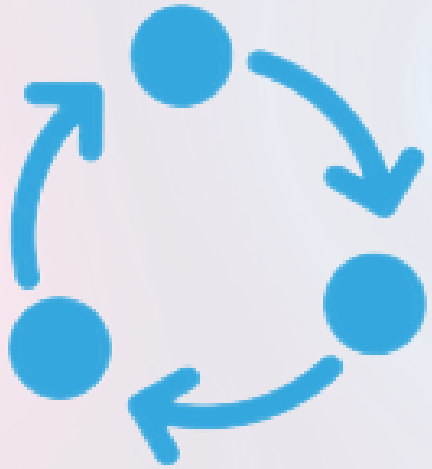






METODOLOGIA

- Conjunto de processos, métodos e ferramentas inter-relacionados
- Conjunto de orientações para aplicar processos, métodos e ferramentas a problemas específicos.



AMBIENTE

O ambiente de um projeto:

- Deve suportar a utilização de determinadas ferramentas e métodos
- Pode impor restrições ao o quê e ao como.



QUALIDADE DE SOFTWARE

- “Qualidade é uma característica ou atributo de alguma coisa” [American Heritage Dictionary]
- “A Qualidade é a ausência de falhas no produto que, através das suas características, satisfaz o cliente e vai ao encontro das suas necessidades” [Juran,1988]
- “Qualidade significa estar em conformidade com os requisitos; é prevenção; significa zero defeitos; deve ser medida pelo custo de não conformidade” [Crosby, 1986]

QUALIDADE DE SOFTWARE

- A Qualidade define-se como a totalidade das propriedades e características de um produto ou serviço que o tornam apto a satisfazer as necessidades implícitas ou explícitas" [ISO-8402]
- "market-driven quality," which is based on achieving total customer satisfaction [IBM]
- The degree to which a set of inherent characteristics fulfills requirements." [ISO9001- 00]

QUALIDADE DE SOFTWARE

- Apesar das várias definições, é consensual que:
  - Os requisitos de software definem as características de qualidade do software e influenciam os métodos de cálculo e critérios de aceitação para avaliar essas características.

QUALIDADE DE SOFTWARE

"Um produto de software apresenta qualidade dependendo do grau de satisfação das necessidades dos clientes sob todos os aspetos do produto." [Sanders, 1994].

- Quanto maior a qualidade, menor o custo que traz.
- No entanto, eliminar erros e defeitos pode trazer custos mais elevados que o próprio desenvolvimento.

QUALIDADE DE SOFTWARE

- A qualidade não deve ser tratada como uma etapa final do projeto. Pelo contrário, deve estar presente ao longo de todo o processo de desenvolvimento, desde a definição de requisitos até à manutenção do software.

- Quando a qualidade é pensada de forma preventiva, reduz-se a probabilidade de defeitos e o esforço de correção em fases posteriores.

QUALIDADE DO PROCESSO VS PRODUTO



QUALIDADE DO PROCESSO VS PRODUTO

- Qualidade do Processo: Forma como o software é desenvolvido.
- Qualidade do Produto: Características e comportamento do software entregue.

- Estas duas perspetivas estão interligadas: um processo mais disciplinado tende a produzir software com maior qualidade.

- Não se deve dissociar o processo do produto. Ainda assim, o foco pode recair mais sobre uma ou outra componente.

CUSTOS DA QUALIDADE

Os custos da qualidade podem ser analisados em quatro grupos:

- custos de prevenção
- custos de avaliação
- custos de falhas internas
- custos de falhas externas

- A deteção tardia de defeitos tende a aumentar o impacto técnico, organizacional e financeiro. Por essa razão, a prevenção e os testes precoces têm elevado valor no desenvolvimento de software.

ERRO E DEFEITO

Erro

- Ocorrência ou condição indesejada que surge durante o processo de desenvolvimento de software e que se afasta da especificação dos seus requisitos.
- Pode ser detetado de imediato ou manter-se até que alguma ação de correção o detete, mas é sempre eliminado antes da entrega ao cliente.

Defeito

- Ocorrência ou condição indesejada que escapa a todas as tentativas (inspeções, revisões, testes e medidas corretivas) de eliminação durante o desenvolvimento e é passado ao cliente.
- O defeito pode nunca ser detetado ou não ser sujeito a correção ou acompanhamento.
- Trivialmente conhecido como **bug**.

TIPOS DE DEFEITOS

- Computacional
- Lógico
- Input e output
- Tratamento de dados
- Interface
- Definição de dados/Base de dados
- Falta de documentação
- Desenho dos requisitos
- Outros

TEST-DRIVEN DESIGN

VITOR MANUEL SANTOS

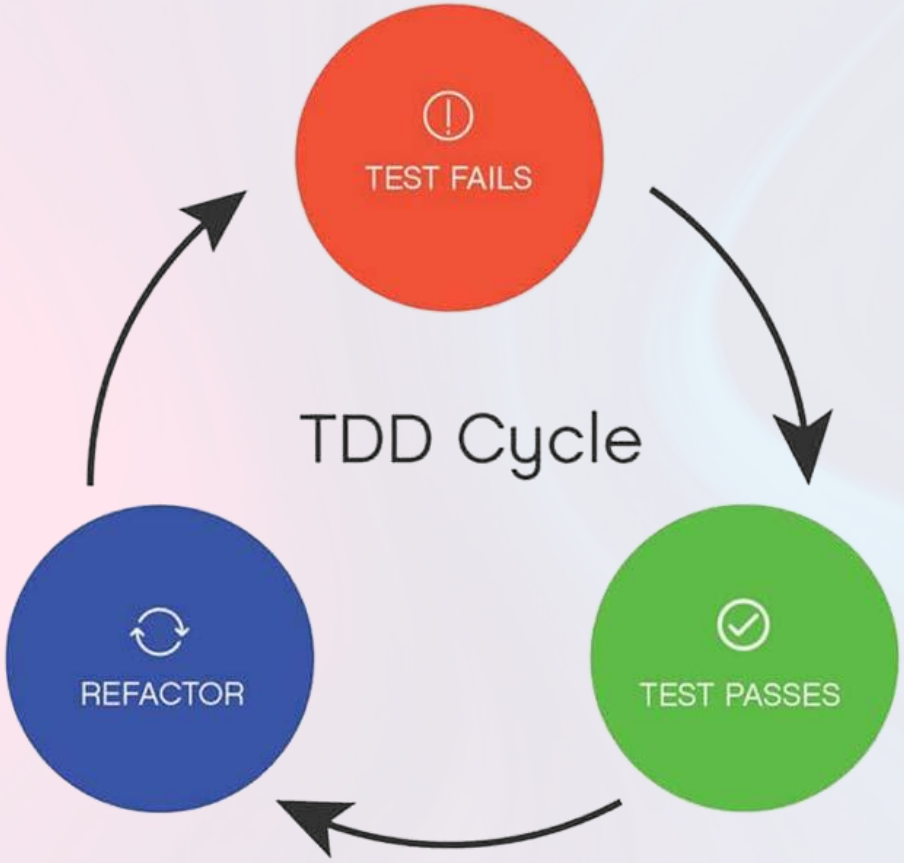
TDD — TEST DRIVEN DEVELOPMENT

- Metodologia de desenvolvimento que prioriza a escrita de testes antes do código de produção.

O processo prevê:

1. Escrever um teste que falha,
2. Desenvolver o código necessário para o teste passar e, por fim,
3. Melhorar o código para atender aos padrões de qualidade.

TDD — TEST DRIVEN DEVELOPMENT



BDD — BEHAVIOR DRIVEN DEVELOPMENT

- Extensão de TDD que realça a necessidade da colaboração entre programadores, QAs e stakeholders não técnicos.

O processo prevê:

1. Definição de comportamentos desejados numa linguagem clara e acessível,
2. Escrita de testes e, então,
3. Desenvolvimento do código.

SOFTWARE QUALITY ASSURANCE

VITOR MANUEL SANTOS

SOFTWARE QUALITY ASSURANCE

- A Garantia da Qualidade do Software é “um conjunto de ações planeadas e sistemáticas, necessárias para assegurar a Qualidade no Software ” [Schulmeyer, 1987]

A Garantia da Qualidade do Software é composta por várias tarefas associadas a dois grupos diferentes:

- Engenheiros de Software, que executam o trabalho técnico
- A Equipa de SQA (Software Quality Assurance), responsável pelo planeamento, acompanhamento, registo de informação, análise e relatórios sobre a Garantia da Qualidade

SOFTWARE QUALITY ASSURANCE

- É possível controlar o que não se pode medir?
- Pode-se medir o que não é possível controlar?





SQA PLAN

- O plano SQA define os meios que irão ser usados para garantir que o software desenvolvido no âmbito de um determinado produto de software satisfaz os requisitos do utilizador e que respeita amplamente as restrições do projeto a um nível de qualidade alto.

Como?

- primeiro é necessário definir e perceber claramente o quality target.
- deve ser considerado: a gestão, o desenvolvimento e os planos de manutenção para o software.
- O standard IEEE730-98, define os detalhes do Software Quality Assurance Plan

SQA PLAN

O que contém o SQA Plan:

- Documentos, standards, práticas e convenções que regem o projeto de modo a controlar e monitorar para garantir a adequação e conformidade.
- Medidas, técnicas (estatísticas) e procedimento para reportar problemas e ações correctivas, recursos (ferramentas, técnicas e metodologias), formação e indicações de como criar documentação específica.
- Outras atividades para garantia de qualidade de software que estejam no SQA Plan, nomeadamente:
  - procurement of supplier software (aquisição de software)
  - commercial off-the-shelf software (COTS)
  - serviços pós-venda

SQA PLAN — ÍNDICE DOCUMENTAL

Secção do SQA Plan	Página
1. Overview	1
1.1 Scope	1
2. References	1
3. Definitions and acronyms	3
3.1 Definitions	3
3.2 Acronyms	3

SQA PLAN — ÍNDICE DOCUMENTAL

Secção do SQA Plan	Página
4. Software Quality Assurance Plan	4
4.1 Purpose (Section 1 of the SQAP)	4
4.2 Reference documents (Section 2 of the SQAP)	4
4.3 Management (Section 3 of the SQAP)	5
4.4 Documentation (Section 4 of the SQAP)	5
4.5 Standards, practices, conventions, and metrics (Section 5 of the SQAP)	7

SQA PLAN — ÍNDICE DOCUMENTAL

Secção do SQA Plan	Página
4.6 Reviews and audits (Section 6 of the SQAP)	7
4.7 Test (Section 7 of the SQAP)	9
4.8 Problem reporting and corrective action (Section 8 of the SQAP)	9
4.9 Tools, techniques, and methodologies (Section 9 of the SQAP)	9
4.10 Code control (Section 10 of the SQAP)	9

SQA PLAN — ÍNDICE DOCUMENTAL

Secção do SQA Plan	Página
4.11 Media control (Section 11 of the SQAP)	9
4.12 Supplier control (Section 12 of the SQAP)	9
4.13 Records collection, maintenance, and retention (Section 13 of the SQAP)	10
4.14 Training (Section 14 of the SQAP)	10
4.15 Risk management (Section 15 of the SQAP)	10

CATEGORIAS DE TRABALHO EM SQA

- A garantia da qualidade de software não se limita à deteção de erros no produto final. Trata-se de um trabalho abrangente, que envolve diferentes dimensões ao longo de todo o ciclo de desenvolvimento.

Dimensões centrais do trabalho em SQA:

- métodos e ferramentas de construção;
- revisões formais;
- estratégia de testes;
- controlo da documentação e do histórico de alterações;
- procedimentos de conformidade com padrões de desenvolvimento;
- mecanismos de medição e análise.

CATEGORIAS DE TRABALHO EM SQA

- Duas áreas tendem a ser subestimadas nas organizações: os testes e os mecanismos de medição. No entanto, ambas são essenciais para uma abordagem pró-ativa à qualidade.

- Nos testes, o erro começa quando o processo é visto como uma etapa secundária. Não basta utilizar ferramentas de automação, é necessário definir uma estratégia adequada, planear os testes e integrá-los no desenvolvimento.

- Na medição, a principal dificuldade está em saber o que medir e como interpretar os resultados. Sem métricas bem definidas, a gestão da qualidade torna-se reativa, baseada apenas em reclamações ou problemas já visíveis.

CATEGORIAS DE TRABALHO EM SQA

- A qualidade de software pode ser analisada através de fatores que nem sempre são observáveis da mesma forma. Alguns podem ser medidos diretamente; outros exigem métodos indiretos de avaliação.

Fatores medidos de forma direta	Fatores medidos de forma indireta
número de defeitos;	usabilidade;
linhas de código;	facilidade de manutenção;
falhas registadas;	satisfação do utilizador;
tempos de execução.	perceção de qualidade.

- A existência destes dois grupos mostra que, para cada contexto, deve ser definido um processo de medição apropriado.

CATEGORIAS DE TRABALHO EM SQA

- O SQA pode ser entendido como um conjunto de subprocessos interligados que sustentam a qualidade ao longo do projeto.

- Componentes principais:
  - testes de software;
  - controlo de qualidade;
  - gestão da configuração de software.

- Estes componentes são suportados por elementos complementares, como:
  - padrões;
  - procedimentos;
  - convenções;
  - especificações.

CATEGORIAS DE TRABALHO EM SQA

- Em conjunto, estes elementos contribuem para disciplinar o desenvolvimento, reduzir variabilidade e promover consistência entre equipas, artefactos e versões.

TESTES NO ÂMBITO DA SQA

Os testes são uma das estratégias mais utilizadas para gestão do risco em projetos de software. O seu principal objetivo é verificar se o produto corresponde aos requisitos definidos.

Contudo, é importante reconhecer que:

- os testes não descobrem todos os defeitos;
- a sua eficácia depende da estratégia adotada;
- fazem parte das atividades de verificação e validação.

Os testes devem, por isso, ser vistos como um mecanismo essencial de confiança, mas não como solução isolada para todos os problemas de qualidade.

QUALITY CONTROL

- O controlo de qualidade incide nos processos e métodos usados para monitorizar o trabalho realizado e os requisitos associados ao produto.
- O seu foco está na identificação e remoção de defeitos antes da entrega ao utilizador, recorrendo a verificações bem definidas no plano de qualidade.

Exemplos de práticas de controlo de qualidade:

- revisões de especificações;
- inspeções de código;
- inspeções de documentação;
- verificações antes da entrega.

GESTÃO DA CONFIGURAÇÃO

A gestão da configuração de software refere-se ao conjunto de atividades usadas para identificar, rastrear e controlar alterações nos elementos de software e do sistema.

O seu objetivo é garantir que:

- os componentes estão corretamente identificados;
- a evolução do sistema é controlada;
- o código e a documentação se mantêm consistentes;
- as versões revistas e testadas correspondem ao estado real do produto.

A gestão da configuração é, assim, uma componente crítica para manter integridade, rastreabilidade e controlo ao longo do ciclo de vida do software.

GESTÃO DA CONFIGURAÇÃO

Em contexto real, a gestão da configuração torna-se indispensável por várias razões:

- o desenvolvimento é realizado em equipa;
- coexistem diferentes versões de software;
- existem várias releases destinadas ao cliente;
- as mudanças precisam de ser controladas sem comprometer a agilidade.

Sem esta disciplina, aumenta o risco de inconsistências, perda de controlo sobre artefactos e dificuldade em perceber o que foi alterado, por quem e com que impacto.

GESTÃO DA CONFIGURAÇÃO

A gestão da configuração envolve um conjunto estruturado de tarefas orientadas para o controlo da mudança.

Tarefas principais:

- identificar os work products sujeitos a alteração;
- estabelecer relações entre esses artefactos;
- definir mecanismos de gestão de versões;
- controlar alterações;
- auditar e reportar modificações.

Estas tarefas permitem assegurar rastreabilidade entre requisitos, documentos, código, testes e entregas.

GESTÃO DA CONFIGURAÇÃO

- Alguns conceitos são essenciais para compreender a gestão da configuração:

- Configuration Item: Elemento de hardware, software ou documentação que necessita de controlo.
- Versão: Estado específico de um determinado item de configuração.
- Baseline: Especificação formalmente revista e aprovada, que serve como referência estável para desenvolvimento futuro.
- Release: Distribuição formal de uma versão aprovada do produto.

- Estes conceitos ajudam a organizar o processo de evolução do software de forma controlada e auditável.

GESTÃO DA CONFIGURAÇÃO

- Na prática, podem ser considerados itens de configuração diversos artefactos técnicos e documentais.

- Exemplos:
  - documento de requisitos;
  - documentos de desenho da solução;
  - plano de testes e restantes documentos de teste;
  - código-fonte;
  - scripts e executáveis;
  - dados de teste;
  - registos de bugs;
  - builds;
  - servidores e componentes de infraestrutura.

BASELINES

- Ao longo do desenvolvimento podem existir diferentes baselines, cada uma representando um ponto de referência formal para o projeto.

- Tipos de baseline referidos:
  - Development Baseline
  - Product Baseline
  - Functional Baseline

- A existência de várias baselines permite estabilizar estados importantes do sistema e suportar a evolução controlada do produto ao longo do tempo.

PROCESSO DA GESTÃO DA CONFIGURAÇÃO

- O processo de gestão da configuração inclui várias etapas articuladas:

- Identificação
  - atribuir nome a cada item;
  - categorizar os itens de configuração.

- Controlo de alterações
  - pedido de alteração;
  - aprovação ou rejeição;
  - checkout;
  - execução da alteração;
  - revisão.

PROCESSO DA GESTÃO DA CONFIGURAÇÃO

- Controlo de versões
  - registo e diferenciação entre versões sucessivas dos artefactos.

- Este processo permite reduzir erros decorrentes de alterações não controladas.

AUDITORIAS E REPORTING NA GESTÃO DA CONFIGURAÇÃO

- As auditorias de configuração têm como finalidade garantir o alinhamento entre o estado real dos artefactos e o plano de configuração definido.

- Estas auditorias procuram:
  - identificar não conformidades;
  - verificar a consistência entre versões e documentação;
  - aplicar medidas corretivas;
  - produzir informação de reporting sobre o estado das configurações.

- O reporting deve tornar claro:
  - quem efetuou a alteração;
  - quando foi realizada;
  - o que foi alterado;
  - qual o estado da auditoria.

TESTES DE SOFTWARE

- Para testar precisamos de perceber para que serve o código e não como ele está ou vai ser implementado.

- Os testes são uma componente central da qualidade de software, mas não a esgotam. A qualidade envolve também processo, revisão, controlo e melhoria contínua.

- No entanto, os testes permitem:
  - detetar defeitos
  - validar comportamentos esperados
  - reduzir regressões
  - apoiar a evolução segura do código

TESTES DE SOFTWARE

VITOR MANUEL SANTOS

VERIFICAÇÃO E VALIDAÇÃO

Conceito	Questão central	Foco
Verificação	Estamos a construir corretamente?	processo / conformidade
Validação	Estamos a construir o que é necessário?	produto / requisitos

- A verificação está mais associada à conformidade com o que foi definido em cada fase.
- A validação procura confirmar se o software satisfaz efetivamente os requisitos e necessidades do utilizador.

TESTES

- Manuais
  - Processo de verificar manualmente as funcionalidades e a interface do utilizador para identificar bugs ou comportamentos inesperados.
  - Os QA seguem cenários de testes pré-definidos ou exploram o sistema livremente para encontrar defeitos.

- Automáticos
  - Uso de software para executar testes e comparar os resultados obtidos com os resultados esperados.
  - Prevê desenvolvimento de scripts de teste que podem ser executados repetidamente sem intervenção manual.

TESTE DE SOFTWARE

- Testes Unitários (Unit Testing):
  - Primeiro nível de teste e realizado durante o desenvolvimento
  - Testes de código-fonte, como funções, métodos ou classes
- Teste de Integração (Integration Testing):
  - Verificação da interação entre módulos ou componentes do software
- Teste de Sistema (System Testing):
  - O software é testado como um todo para garantir que atende aos requisitos funcionais de negócio
  - É simulado comportamento do software num ambiente semelhante ao de produção
- Teste de Aceitação (Acceptance Testing):
  - Determinam se o software atende aos critérios, necessidades e expectativas estabelecidos pelos stakeholders, incluindo o cliente ou utilizador final.

TESTE DE SOFTWARE

- O objetivo é garantir que o software esteja pronto para ser implantado num ambiente real.

TESTES UNITÁRIOS

- Quem faz: Developers

- Dicas:
  - Testes curtos, claros, objetivos
  - Pensar em todas as possibilidades, mas sem repetir
  - Não ignorar nenhum! Testar a lista inteira.

TESTES UNITÁRIOS

- Os testes unitários permitem validar pequenas unidades de código de forma rápida, repetível e objetiva. São particularmente úteis durante o desenvolvimento, pois ajudam a detetar problemas cedo e facilitam a evolução segura do software.

- Entre as principais vantagens, destacam-se:
  - maior confiança no código produzido
  - apoio ao refactoring
  - deteção precoce de defeitos
  - melhor documentação do comportamento esperado
  - redução de regressões

TESTES DE INTEGRAÇÃO

- Estes erros são mais comuns de acontecer em grandes equipas que trabalham em simultâneo no mesmo projeto

- Têm como objetivos:
  - Encontrar falhas na integração e comunicação dos módulos e componentes
  - Testar a dependência dos componentes

TESTES DE SISTEMA

- São feitos quando o produto já está desenvolvido
- Servem para garantir qualidade dos componentes
- Devem ser usadas ferramentas eficazes (criando listas bastante completas)
- Aqui, podem ser identificados:
  - Testes funcionais positivos (o que o programa faz) e negativos (o que o programa não deve fazer)
  - Testes não funcionais (validações de negócio e particulares)
  - Testes de usabilidade
  - Testes de performance
  - Testes de regressão (verificação de novas versões)
  - Testes de segurança

TESTES DE SISTEMA

- Efetuados numa perspetiva de utilizadores finais
- Dicas:
  - Usar ferramentas eficazes (criando listas bastante completas)

TESTES DE ACEITAÇÃO

- Voltados para funcionalidades numa perspetiva dos utilizadores finais
- De preferência, feitos pelo PO ou responsável pelo requisito

- São os grandes grupos dos testes de sistema
- Sempre que possível, feitos num ambiente específico para testes e próximo do real (ambiente de homologação)

PLANO DE TESTES

- Segundo a norma ANSI/IEEE 829, o plano de testes é o documento que define o âmbito, a abordagem, os recursos e o escalonamento das atividades de teste. Identifica os itens a testar, as funcionalidades abrangidas, as tarefas a executar, os responsáveis e os riscos que podem exigir medidas de contingência.

PLANO DE TESTES

- Um bom plano de testes ajuda a organizar e gerir o esforço de teste de forma estruturada. Permite clarificar o que será testado, com que abordagem, em que momento e com que recursos. Em projetos mais exigentes, o plano deixa de ser apenas apoio operacional e passa a constituir um produto documental relevante para o próprio projeto.

PLANO DE TESTES — PRODUTO OU FERRAMENTA?

- Do ponto de vista do cliente, o mais importante é que o software funcione corretamente. Por isso, o plano de testes raramente é visto como um produto final com valor direto. Ainda assim, para a equipa de desenvolvimento e teste, trata-se de uma ferramenta essencial, porque apoia a gestão do processo, a coordenação das tarefas e a deteção sistemática de falhas.

PLANO DE TESTES — DOCUMENTAÇÃO

- A norma IEEE 829 não se limita ao plano de testes. O processo documental inclui também elementos como:
  - especificação da conceção dos testes;
  - especificação dos casos de teste;
  - procedimentos de teste;
  - registos de execução;
  - relatórios de teste;
  - especificações de entradas e saídas;
  - notas sobre dependências entre casos de teste.

PLANO DE TESTES — DOCUMENTAÇÃO

- Além da documentação técnica, o plano de testes deve considerar:
  - documentos a produzir após os testes;
  - planeamento temporal;
  - planeamento de recursos humanos;
  - responsabilidades de cada elemento da equipa;
  - critérios para suspensão e retoma dos testes.
  - Isto mostra que o plano de testes não serve apenas para “testar”, mas também para coordenar o trabalho de teste.

PLANO DE TESTES — DOCUMENTAÇÃO

- Entre as secções nucleares de um plano de testes destacam-se:
  - identificador do plano;
  - introdução;
  - itens de teste;
  - características a testar;
  - características que não serão testadas.

- Estas secções ajudam a delimitar claramente o alcance do esforço de teste e a evitar ambiguidades quanto ao que está ou não incluído.

PLANO DE TESTES — DOCUMENTAÇÃO

- Um plano de testes deve ainda explicitar:
  - a abordagem adotada;
  - quem executa os testes;
  - técnicas e ferramentas usadas;
  - critérios de sucesso e insucesso;
  - critérios de suspensão e retoma;
  - produtos gerados pelos testes;
  - tarefas necessárias à preparação e execução;
  - ambiente de teste necessário.

PLANO DE TESTES — DOCUMENTAÇÃO

- Na parte final do plano, é importante prever:
  - responsabilidades;
  - necessidades de recursos humanos e formação;
  - escalonamento e datas-chave;
  - riscos e contingências;
  - aprovações formais.

- Desta forma, o plano de testes torna-se um instrumento de gestão, comunicação e controlo.

PLANO DE TESTES — DOCUMENTAÇÃO

- O desenvolvimento inicial do material de teste deve incluir:
  - testar com base na documentação existente;
  - organizar a informação do sistema para facilitar testes eficientes;
  - identificar funções relevantes;
  - realizar uma análise simples de limites;
  - testar valores fronteira sempre que existam dados com intervalos ou restrições.
- Estas práticas ajudam a construir casos de teste mais sólidos desde cedo.

PLANO DE TESTES — DOCUMENTAÇÃO

- Nem todas as áreas do sistema devem receber o mesmo nível de atenção. O esforço de teste deve incidir sobretudo em:
  - erros mais prováveis;
  - erros mais visíveis para o utilizador;
  - áreas mais usadas do sistema;
  - funcionalidades distintivas;
  - áreas mais difíceis de corrigir;
  - zonas do programa melhor compreendidas pela equipa.

PLANO DE TESTES — BOAS PRÁTICAS

- Algumas recomendações essenciais:
  - testar o mais cedo possível;
  - escrever casos de teste antes da execução;
  - aplicar os testes em vários níveis;
  - automatizar quando fizer sentido;
  - definir saídas esperadas com base nas especificações e não no código;
  - procurar independência e espírito crítico na atividade de teste.
- Estas práticas reforçam a qualidade do processo e não apenas do produto final.

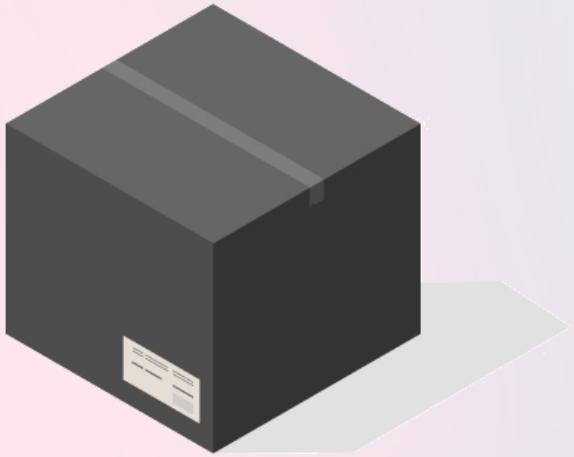
ESTRATÉGIAS DE TESTE

- As estratégias de desenho de casos de teste apresentadas nesta parte do material são duas:
  - caixa preta
  - caixa branca

- Ambas procuram revelar defeitos, mas partem de perspetivas diferentes sobre o software a testar.

ESTRATÉGIAS DE TESTE

QA testers



Black box - we do not know anything

Developers



White box - we know everything

TESTES DE CAIXA PRETA

- Os testes de caixa preta, também designados testes funcionais, avaliam o comportamento externo do software sem considerar a sua estrutura interna. O desenho dos casos de teste baseia-se nas entradas e saídas esperadas, com o objetivo de verificar se os requisitos e especificações foram corretamente implementados.

TESTES DE CAIXA BRANCA

- Os testes de caixa branca, ou testes estruturais, avaliam o comportamento interno dos componentes de software. São especialmente adequados a testes unitários, porque exigem conhecimento da lógica interna, das decisões, dos fluxos e da estrutura do programa. O objetivo é verificar se os elementos lógicos e de dados estão a funcionar adequadamente.

TESTES DE CAIXA PRETA VS BRANCA

- A abordagem de caixa preta pode ser aplicada a componentes pequenos ou grandes, porque se centra no comportamento observável.

- Já a abordagem de caixa branca é mais apropriada para componentes pequenos, uma vez que o nível de detalhe necessário para desenhar os testes aumenta significativamente com a complexidade do código.

CAIXA PRETA E CAIXA BRANCA

Critério	Black box
Perspetiva	Entradas e saídas observáveis Estrutura interna oculta
Fontes de conhecimento	Requirements document Specifications Domain knowledge Defect analysis data
Métodos	Equivalence class partitioning Boundary value analysis State transition testing Cause and effect graphing Error guessing

CAIXA PRETA E CAIXA BRANCA

Critério	White box
Perspetiva	Estrutura interna conhecida Fluxos e decisões do programa
Fontes de conhecimento	High-level design Detailed design Control flow graphs Cyclomatic complexity
Métodos	Statement testing Branch testing Path testing Data flow testing Mutation testing Loop testing

TESTES DE CAIXA PRETA

- Os testes de caixa preta procuram descobrir defeitos relacionados com:
  - funções incorretas ou ausentes;
  - erros de interface;
  - erros em estruturas de dados ou no acesso a bases de dados externas;
  - problemas de inicialização;
  - problemas de término.
- São, por isso, particularmente úteis para validar o comportamento funcional do sistema.

TESTES DE CAIXA PRETA

- Perguntas orientadoras em testes de caixa preta
- Ao aplicar esta estratégia, o tester deve procurar responder a questões como:
  - como validar funcionalmente o sistema?
  - que classes de entrada geram bons casos de teste?
  - a que valores o sistema é mais sensível?
  - onde estão as fronteiras das classes de dados?
  - que volumes de dados o sistema suporta?
  - que efeitos podem surgir de combinações específicas de dados?

ECP — EQUIVALENCE CLASS PARTITIONING

- A Partição em Classes de Equivalência é uma técnica de teste de caixa preta que divide o domínio de entrada em grupos de valores com comportamento esperado semelhante.

- Em vez de testar todos os valores possíveis, seleciona-se um valor representativo de cada classe.

- Ideia principal:
  - reduzir o número de testes;
  - manter cobertura relevante;
  - considerar classes válidas e inválidas.

ECP — EXEMPLOS DE CLASSES

Situação ilustrada	Classes / grupos exibidos
Para uma operação sqrt(x):	classe válida: $x \geq 0$; classe inválida: $x < 0$
Para uma função de valor absoluto:	valores negativos; zero e positivos; entradas inválidas, como tipos não inteiros.

Cada classe deve originar pelo menos um caso de teste representativo.

ECP — MATRIZ E CASOS DE TESTE

Critério	Classe de Equivalência válida	Classe de equivalência inválida
nº de entradas (inputs)	1	0 , >1
tipo de entradas	inteiro	não inteiro
valor específico x	< 0 , >= 0	

Casos de testes baseados nas classes de equivalência especificadas:

ECP — MATRIZ E CASOS DE TESTE

Caso apresentado	Indicadores de classe assinalados na imagem
X = -10	primeiro, segundo e terceiro indicadores das classes válidas
x = 100	primeiro, segundo e terceiro indicadores das classes válidas
x =	indicador de número de entradas inválido
x = 10 20	indicador de número de entradas inválido
x = "XYZ"	indicador de tipo de entrada inválido

BVA – BOUNDARY VALUE ANALYSIS

- A Análise de Valor Limite é uma técnica que foca os testes nos extremos do domínio de entrada e nos valores imediatamente próximos desses extremos.

- Isto acontece porque muitos defeitos tendem a surgir junto aos limites.

- Devem ser testados:
  - valor mínimo;
  - valor máximo;
  - valor imediatamente abaixo do mínimo;
  - valor imediatamente acima do máximo.